

Desafio Técnico — Desenvolvedor Sênior Java

Entrega: Repositório Git público (GitHub/GitLab) com README de instruções para rodar

Contexto

Você foi contratado para construir um microserviço de **gestão de agendamentos domiciliares**. Neste desafio, você deve implementar a funcionalidade de **criação e listagem de agendamentos**, seguindo os princípios de **Clean Architecture (Hexagonal)**.

O que deve ser implementado

Entidade de Domínio — Agendamento

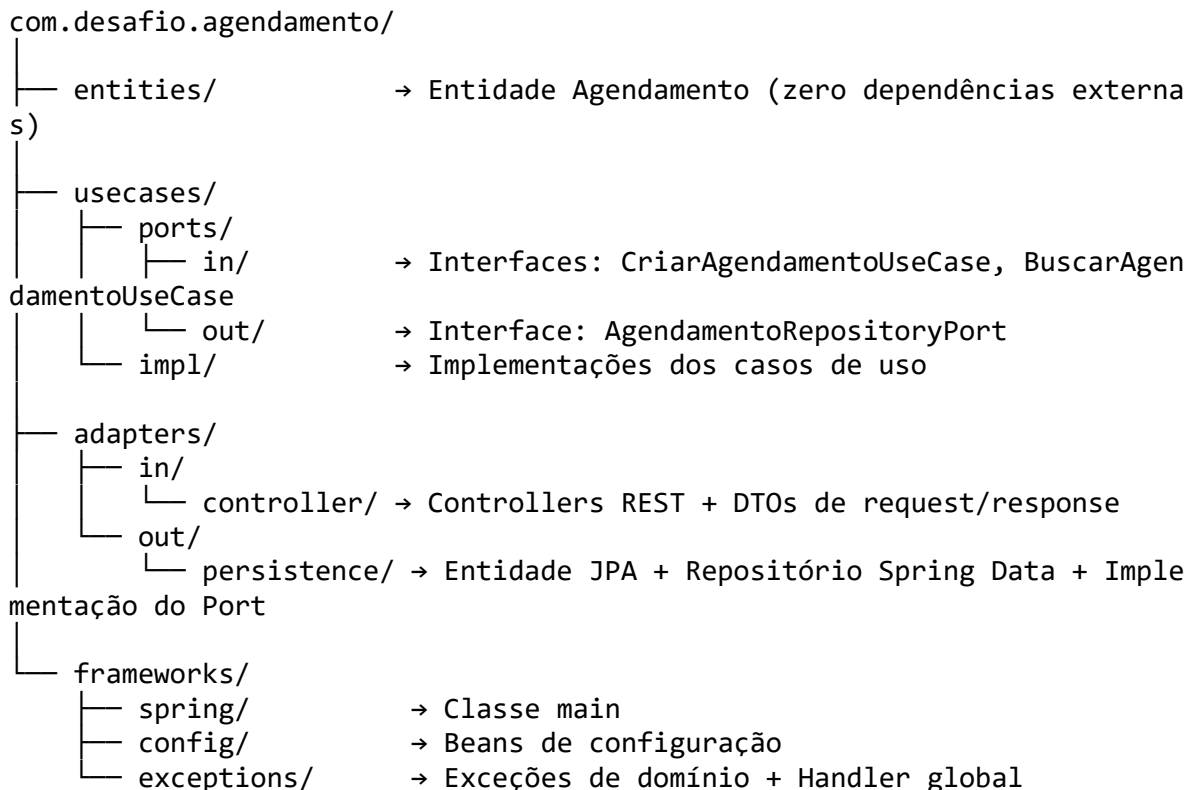
Campo	Tipo	Regra
id	UUID	Gerado automaticamente
pacienteNome	String	Obrigatório, mín. 3 caracteres
pacienteCpf	String	Obrigatório, formato válido
dataAgendamento	LocalDateTime	Obrigatório, não pode ser no passado
status	Enum	PENDENTE, CONFIRMADO, CANCELADO
observacao	String	Opcional
criadoEm	LocalDateTime	Preenchido automaticamente

Endpoints REST obrigatórios

POST	/api/v1/agendamentos	→ Criar agendamento
GET	/api/v1/agendamentos	→ Listar todos (filtro opcional por status)
GET	/api/v1/agendamentos/{id}	→ Buscar por ID
PATCH	/api/v1/agendamentos/{id}/status	→ Atualizar status

Requisitos Arquiteturais

O projeto **obrigatoriamente** deve seguir a estrutura de pacotes abaixo:



Atenção: Entidades de domínio e casos de uso devem seguir os princípios da arquitetura hexagonal.

Requisitos Técnicos

- **Java 21+** (25 é um diferencial)
 - **Spring Boot 3.x** (4 é um diferencial)
 - **Spring Data JPA + H2** em memória (sem necessidade de Docker)
 - **Maven ou Gradle**
 - Validações com **Bean Validation** (@Valid)
 - Tratamento de erros centralizado com **@RestControllerAdvice**
 - Respostas padronizadas (envelope com data, message, timestamp)
 - **Testes unitários** nos casos de uso com JUnit 5 + Mockito
 - Documentação via **SpringDoc OpenAPI** (disponível em /swagger-ui.html)
-

Regras de Negócio

1. Não é possível criar um agendamento com dataAgendamento no passado.
 2. Não é possível alterar o status de um agendamento CANCELADO.
 3. O CPF deve ter formato válido (apenas dígitos, 11 caracteres — validação simples é suficiente).
 4. Ao cancelar um agendamento, o campo observacao deve ser informado obrigatoriamente.
-

Diferenciais (não obrigatórios)

- Publicar um evento em um tópico Kafka ao criar o agendamento (agendamento-criado)
 - Usar record do Java para os DTOs de request/response
 - Adicionar testes de integração com @SpringBootTest
 - Implementar paginação na listagem
 - Usar @ConfigurationProperties para alguma configuração do projeto
-

Entrega esperada

O repositório deve conter:

1. **Código-fonte** seguindo a estrutura de pacotes exigida
 2. **README.md** com:
 - Como rodar o projeto
 - Exemplos de requisições (cURL ou link do Swagger)
 - Decisões arquiteturais tomadas e por quê
 3. Pelo menos **3 testes unitários** nos casos de uso
-

Qualquer dúvida sobre o desafio, entre em contato. Boa sorte!